ALGORISME COCKE-KAMASI-YOUNGER

Programació i Algorismia Avançada

Mariona Casasnovas Simon i Núria López Encinas

15 de juny de 2025

Grau en Intel·ligència Artificial

Facultat d'Informàtica de Barcelona

UPC

ÍNDEX

1. Introducció
2. Enumeració dels fitxers entregats
3. Algorisme CKY determinista
    a. Funcionament de l'algorisme principal
4. Extensions
    a. Conversió de CFG a CNF
    b. Algorisme CKY probabilistic
5. Jocs de proves
    a. CKY
        i. Prova 1
        ii. Prova 2
        iii. Prova 3
        iv. Prova 4
        v. Prova 5
    b. CNF
        i. Prova 1
        ii. Prova 2
        iii. Prova 3
        iv. Prova 4
        v. Prova 5
        vi. Prova 6
        vii. Prova 7
    c. PCKY
        i. Prova 1
        ii. Prova 2
        iii. Prova 3
6. Conclusions

1. Introducció

Aquesta pràctica té com a objectiu posar en pràctica els coneixements sobre programació dinàmica mitjançant la implementació de l'algorisme determinista CKY (Cocke–Kasami–Younger), un algorisme clàssic per l'anàlisi sintàctic de cadenes segons una gramàtica incontextual (CFG) en forma normal de Chomsky (CNF), el qual permet determinar si una paraula pertany al llenguatge generat per una gramàtica donada.

El desenvolupament d'aquesta inclou la lectura de fitxers d'entrada, l'anàlisi de cadenes mitjançant CKY i la validació del funcionament amb diversos jocs de proves. A més, s'han considerat les dues extensions proposades: la conversió automàtica de CFG a CNF i l'adaptació de l'algorisme a una versió probabilística.

2. Enumeració dels fitxers entregats

a. *CKY.py*

Conté la funció que aplica l'algorisme CKY (Cocke–Kasami–Younger). Reb com entrada la gramàtica representada per un diccionari de regles, el símbol inicial i el conjunt de símbols terminals. I retorna un booleà que indica si la paraula és generada o no per la gramàtica donada. Un exemple del format de la gramàtica seria aquest:

```
gramàtica = ({
                'S': [('A','B')],
                'A': [('B','C'),('a',),()],
                'B': [('C','C',),('b',)],
                'C': [('c',)],
              },'S',{'a','b'})
```

Les claus del diccionari són les strings que representen els símbols no terminals, i els valors són llistes de les produccions, on cada producció és una tupla per cada símbol que genera. Si només genera un únic símbol s'indica que és una tupla amb una coma al final. La producció buida és representa amb la tupla buida.

b. *CFGtoCNF.py*

Conté la funció que transforma qualsevol gramàtica incontextual (CFG) a forma normal de Chomsky (CNF). Reb com entrada una gramàtica representada per un diccionari de regles, el símbol inicial i el conjunt de símbols terminals. I retorna la transformació d'aquesta gramàtica a CNF representada de la mateixa forma.

c. *PCKY.py*

Conté la funció que aplica l'algorisme CKY probabilístic (Probabilistic Cocke–Kasami–Younger). Reb com entrada la gramàtica representada per un diccionari de regles, on cada regla conté la seva probabilitat, el símbol inicial i el conjunt de símbols terminals. I retorna un booleà que indica si la paraula és generada o no per la gramàtica donada i la probabilitat d'aquesta. El format de

la gramàtica és el mateix que abans però ara les produccions son una tupla de la tupla dels símbols produits i la seva probabilitat. Per exemple: *'B': [(('A','X'),0.3),(('b'),0.4),(('a'),0.3)]*

d. *entrada.txt*

És un fitxer de text amb tots els jocs de prova. Les gramàtiques estan representades de la mateixa forma que hem vist a classe i per cada gramàtica hi ha una llista de paraules amb les que provar-la. El format dels jocs de prova consisteix en un títol per cada apartat seguit dels diferents jocs de prova de l'apartat i acabat amb 'fi'. Cada joc de proves consisteix en un títol que s'utilitzarà a l'hora d'imprimir els resultats, una llista de les paraules que es volen provar, el símbol inicial de la gramàtica, els símbols terminals escrits entre claudàtors i finalment les regles de la gramàtica en el mateix format que el explicat a classe, on '>' representen les fletxes i 'eps' representa epsilon. finalment s'indica el fi de la gramàtica amb 'fi'. Per exemple:

4.4

[ ]
S [a b]
S > AB
A > AB | a
B > b | eps
fi

En el cas del probabilístic s'afegeixen les probabilitats al costat de cada producció sense parèntesis: *S > AA 0.7 | c 0.2 | eps 0.1*

e. *lectura.py*

És una funció dissenyada per llegir l'entrada de jocs de prova i transformar-los al format desitjat: un diccionari de regles, el símbol inicial i el conjunt de terminals. Llegeix l'entrada mitjançant pytoker i guarda un diccionari de jocs de prova per cada algoritme.

f. *Jocs_de_prova.py*

És un programa que llegeix l'entrada donada de jocs de prova mitjançant la funció de *lectura.py* i els prova per cada algoritme imprimint els resultats de forma entenedora.

3. Algorisme CKY determinista

L'algorisme CKY (Cocke–Kasami–Younger) és algorisme de programació dinàmica que permet determinar si una paraula pot ser generada per una gramàtica incontextual en Chomsky Normal Form (CNF). Funciona omplint una taula (que acabarà tenint una forma triangular) on cada cel·la representa les possibles variables que poden generar un segment concret de la cadena. A partir de les produccions de la gramàtica i combinant subcadenes més petites, l'algorisme construeix possibles derivacions fins arribar al símbol inicial. Si aquest apareix a la casella superior de la taula, la cadena pertany al llenguatge.

a. Funcionament de l'algorisme principal

L'algorisme CKY s'implementa com un procés de programació dinàmica que construeix una taula triangular per determinar si una paraula és generada per una gramàtica en Chomsky Normal Form (CNF). Aquesta taula conté, a cada cel·la, el conjunt de símbols no terminals que poden derivar la subcadena corresponent de la paraula.

L'algoritme comença creant dos diccionaris invertits de les regles; és a dir, on les claus són les produccions i els valors són tots els símbols que les generen, un per les produccions no terminals i un per les terminals. S'ha decidit implementar-ho així per evitar recórrer totes les regles quan es cerca una producció específica, optimitzant així el procés d'omplir la taula dinàmica.

En la implementació realitzada, la construcció de la taula comença per la primera columna (és a dir, aquelles cel·les que corresponen a subcadenes d'un únic símbol terminal). Per a cada símbol de la paraula, es recullen totes les regles unàries de la gramàtica que poden generar-lo directament. Aquestes s'insereixen a la primera columna de cada fila.

La taula es va omplint de forma ascendent, des de la segona columna fins a la superior dreta, que correspon a la paraula completa. Per a cada cel·la, es consideren totes les possibles formes de dividir la subcadena corresponent en

dues parts més petites. Es combinen els símbols no terminals ja registrats en les cel·les inferiors i s'analitza si alguna regla binària de la gramàtica pot generar la combinació obtinguda. Si és així, el símbol resultant s'afegeix a la cel·la actual.

Aquest procediment continua de manera sistemàtica fins a arribar a la part superior de la taula. Finalment, es verifica si el símbol inicial de la gramàtica es troba a la cel·la que representa tota la paraula (la superior dreta). Si és així, s'accepta que la paraula pot ser generada per la gramàtica. A la taula només es guarden els símbols que generen les produccions perquè sigui més senzill, però si es guardes la regla completa després es podria reconstruir l'arbre que genera la paraula molt facilment.

<u>Anàlisi del codi</u>

L'algorisme CKY (Cocke-Kasami-Younger) s'ha implementat com una funció amb l'objectiu de determinar si una paraula concreta pot ser generada per una gramàtica en CNF. La funció rep com a entrada una paraula i una gramàtica representada com una tupla amb tres components: el diccionari de produccions (gram), el símbol inicial (inicial) i la llista de terminals (terminals).

Es recorren totes les produccions de la gramàtica per generar els dos diccionaris de regles invertides. Si $|G|$ és el nombre total de produccions, aquest pas és $\Theta(|G|)$.

La taula es construeix com una matriu triangular de dimensions $n \times n$, on n és la longitud de la paraula d'entrada. Aquesta taula conté, a cada cel·la, les variables no terminals que poden generar el subsegment corresponent de la paraula.

Si la paraula és la paraula buida es comprova directament si aquesta és generada pel símbol inicial, si és així retorna True, en cas contrari retorna False.

L'algorisme comença emplenant la primera columna de la matriu, que correspon als símbols terminals de la paraula. A continuació, es construeixen iterativament les cel·les restants, considerant totes les particions possibles dels subsegments i combinant les categories sintàctiques que poden aparèixer a cada meitat d'una partició mitjançant el producte cartesià. Per a cada combinació, es comprova si alguna de les regles de la gramàtica genera aquest parell de símbols utilitzant els diccionaris invertits que s'han creat a l'inici. Si és així, s'afegeix la variable corresponent a la cel·la. S'han d'omplir n22 cel·les, i per cada cel·la es fan fins a n particions, on cada una pot tenir fins a K no terminals (on K és l'alfabet de no terminals). Per tant seria Θ(n3*K2), si considerem K com una constant que representa la mida de la gramàtica queda Θ(n3)

Finalment, l'algorisme comprova si el símbol inicial de la gramàtica apareix a la cel·la superior dreta de la taula (taula[0][n-1]), la qual representa tota la paraula. Això indica que la paraula pot ser generada per la gramàtica.

Per tant, la implementació assoleix la complexitat asimptòtica $\Theta(n^3 + |G|)$ on n és la longitud de la paraula i $|G|$ el nombre de regles de la gramàtica.

4. Extensions

A l'enunciat de la pràctica es plantegen dos possibles propostes per implementar de forma opcional.

a. Conversió de CFG a CNF

La primera extensió proposada a l'enunciat de la pràctica consisteix a implementar un programa que transformi qualsevol gramàtica incontextual (CFG) a forma normal de Chomsky (CNF). D'aquesta manera es podrà fer servir l'algorisme CKY amb qualsevol gramàtica incontextual.

Per la seva implementació s'ha creat una funció d'un únic paràmetre que representa la gramàtica: una tupla de tres elements amb el diccionari de regles, el símbol inicial i una llista amb els terminals. A la funció es realitzen 5 canvis o comprovacions per que la gramàtica estigui en CNF.

1. Creació d'un nou símbol inicial
   Primer es comprova si alguna regla produeix el símbol inicial. En aquest cas, s'afegeix un nou símbol inicial S_0 que deriva el símbol inicial original, per evitar ambigüitats i facilitar les transformacions posteriors. És a dir, es crea la regla $S_0 > S$ i S_0 es converteix en el nou símbol inicial. Aquesta regla afegida és unitària, incomplint les normes de CNF, però serà tractada als següents passos.

2. Eliminació de produccions nules (epsilon-produccions)
   S'implementa un procés iteratiu per eliminar les produccions que generen la cadena buida (ε). Primer de tot es detecten totes les regles que generen la paraula buida, i a partir d'aquestes es van cercat totes les que la generen indirectament de forma iterativa fins que les troba totes. Un cop les té totes, genera totes les regles que es poden derivar eliminant els símbols que generen la producció buida, tenint en compte de no afegir regles duplicades ni autoproduccions (A>A).

3. Eliminació de produccions unitàries

   Les produccions unitàries, és a dir, aquelles on un no terminal deriva un altre únic no terminal (ex. A → B), es substitueixen per les produccions del no terminal derivat. Això es fa cercant les produccions unitàries de forma iterativa, i cada cop que s'afegeix una producció nova es mira si és unitària, per assegurar-se que s'eliminen totes.

4. Descomposició de produccions llargues

   Les produccions amb més de dos símbols al costat dret es descomponen en una cadena de produccions binàries introduint nous símbols no terminals (denominats amb números com a 0, 1, etc.).

   Per exemple, una producció A → B C D es transforma en A → B X_0 i X_0 → C D.

   Aquesta conversió es fa de manera repetida fins que totes les produccions compleixin la restricció de tenir com a màxim dos símbols. Si una producció té més de dos símbols es crea un nou símbol que genera els dos primers, i no genera mai dues normes iguals. Per no repetir els noms de les variables es crea un comptador i que comença a 0 i va sumant 1 per gada regla afegida, i la nova norma es defineix com X_i, abans d'afegir la regla comprova que no existeix el símbol per si de cas s'havia utilitzat en la gramàtica original.

5. Separació de terminals i no terminals

   Quan apareixen terminals en produccions amb més d'un símbol, aquests terminals es substitueixen per no terminals nous que produeixen només aquell terminal.

   Per exemple, una producció A → a B es converteix en A → X_0 B i X_0 → a.

   Aquest pas assegura que els terminals no apareguin barrejats amb altres símbols dins les produccions binàries, com requereix la CNF. Els nous símbols afegits es continuen del comptador anterior i també

verifica que no existeixi anteriorment i que no es generi la mateixa norma més d'un cop.

b. Algorisme CKY probabilistic

Aquesta extensió és una implementació del CKY adaptada a gramàtiques probabilístiques i s'ha implementat utilitzant el CKY creat anteriorment però amb noves funcionalitats perquè calculi la probabilitat de la paraula.

L'algoritme comença, igual que el CKY, creant els diccionaris de regles invertides, però en aquest cas guarda també la probabilitat de cada una.

Segueix les mateixes passes que el CKY amb l'únic canvi que calcula també la probabilitat de cada un multiplicant la probabilitat de les dues divisions per la probabilitat de la regla que les uneix. Si es pot generar amb la mateixa regla de dues formes diferents guarda sempre la de major probabilitat.

Al final es mira si el símbol inicial apareix a la cel·la final i recuperem la seva probabilitat, que retornem juntament amb el booleà d'acceptació.

5. Jocs de proves

Per provar els programes s'han creat un conjunt de jocs de prova dissenyats per validar cada funcionalitat dels programes i verificar el seu comportament correcte. S'ha creat un fitxer d'entrada *entrada.txt* , un programa que permet llegir-la *lectura.py* i un programa que prova els jocs de prova de l'entrada amb els algoritmes creats.

a. CKY

S'han dissenyat un conjunt de jocs de prova per la implementació de l'algoritme CKY que pretenen comprovar el seu funcionament i validesa. Cada joc de prova es centra en validar una funcionalitat o un cas especial.

i. Prova 1

El primer joc de proves serveix per verificar els casos bàsics de l'algorime

gramàtica:

S > AB | c

A > a

B > b

paraules: *'c', 'd', '', 'ab', 'a', 'ba' i 'aba'*

Paraula	***Booleà***	***Taula CKY***	***Observacions***
'c'	*True*	*[['S']]*	*Reconeix el cas base S > c*
'd'	*False*		*Detecta que 'd' no és un símbol terminal de la gramàtica i, per tant no el genera*
''	*False*		*Detecta que el símbol inical no genera la paraula buida, per tant aquesta no serà part del llenguatge*
'ab'	*True*	*[['A'], ['S']]* *[['B']]*	*Comprova que reconeix la derivació binària*
'a'	*False*	*[['A']]*	*Comprova que retorna False si la regla no és la inicial*
'ba'	*False*	*[['B'], []]* *[['A']]*	*Comprova que l'algoritme té en compte l'ordre de les regles.*
'aba'	*False*	*[['A'], ['S'], []]*	*Comprova que encara que s'arribi al símbol inical,*

		[['B'], []] *[['A']]*	*no és dona per bo si no és la última casella.*

ii. Prova 2

El segon joc de proves pretén comprovar que l'algoritme detecta totes les possibles regles en cada passa i les té totes en compte. El que esperem de la sortida no és només que trobi la regla S a l'última casella de la taula, sinó que també trobi C. També estem comprovant que detecti la paraula buida si la gramàtica la reconeix.

gramàtica:

S > AB |

C > AB

A > a

B > b

paraules: *'ab', ''*

Paraula	***Booleà***	***Taula CKY***	***Observacions***
'ab'	*True*	*[['A'], ['C', 'S']]* *[['B']]*	*Té en compte les dues regles que deriven AB*
''	*True*		*Reconeix que el símbol inicial genera la paraula buida*

iii. Prova 3

Amb aquest joc de proves es pretén comprovar que no s'afageix la mateixa regla dos cops en una casella de la taula.

gramàtica:

S > AA | BB

A > a

B > a

paraules: *'aa'*

Paraula	***Booleà***	***Taula CKY***	***Observacions***

'aa'	*True*	*[['A', 'B'], ['S']]* *[['A', 'B']]*	*Només s'afegeix un cop la regla S*

iv. Prova 4

Aquest joc de proves serveix per comprovar que l'algoritme calcula correctament el producte cartesià per cada casella de la taula i genera totes les possibles parelles de símbols. Per això s'ha generat el següent cas:

		x
	E F G	C D
A B	Z	H I J
a	z	b

Es vol comprovar que a la casella marcada amb 'x' es tenen en compte totes les produccions: AC, AD, BC, BD, EH, EI, EJ, FH, FI, FJ, GH, GI, GJ i no es tenen en compte cap més. Per això s'ha creat la gramàtica corresponent per generar la taula d'adalt i la qual té una regla per cada possible parella per assegurar-se que no les afegeix a la casella 'x'.

gramàtica:

A > a	*E >AZ*	*'AC' > AC*
B > a	*F > BZ*	*'AD' > AD*
H > b	*G > AZ*	*'BC' > BC*
I > b	*C > ZI*	...
J > b	*D > ZJ*	*'AH' > AH*
Z > z		*'AI' > AI*
		'AJ' > AJ
		'AE' > AE
		...
		'AB' > AB
		'BA'> BA
		'EF' > EF
		...

paraules: *'azb'*

Paraula	***Booleà***	***Taula CKY***	***Observacions***
'azb'	*False*	*[['A', 'B'], ['E', 'F', 'G'], ['AC', 'AD', 'BC', 'BD', 'EH', 'EI', 'EJ', 'FH', 'FI', 'FJ', 'GH', 'GI', 'GJ']]* *[['Z'], ['C', 'D']]* *[['H', 'I', 'J']]*	*Els resultats de la última casella son els que s'esperaven*

v. Prova 5

Finalment s'ha provat amb una gramàtica un poc més complexa per comprovar la capacitat de l'algorisme de gestionar més produccions i regles alhora.

gramàtica:

S > ZZ | AB | DE | EF | UU |

Z > AB | DE | EF | UU

A > a

B > b

C > c

D > BC

E > CA

F > AB

U > DF

X > a

paraules: 'caab', 'caba', 'bcabbcab'

Paraula	***Booleà***	***Taula CKY***	***Observacions***
'caab'	*True*	*[['C'], ['E'], [], ['S', 'Z']]* *[['A', 'X'], [], []]* *[['A', 'X'], ['F', 'S', 'Z']]* *[['B']]*	*Troba la solució correcta*
'caba'	*False*	*[['C'], ['E'], [], []]* *[['A', 'X'], ['F', 'S', 'Z'], []]* *[['B'], []]* *[['A', 'X']]*	*És capaç de reconèixer que la paraula no pot ser generada per la gramàtica*
'bcabbcab'	*True*	*[['B'], ['D'], [], ['U'], [], [], [], ['S', 'Z']]*	*Troba la solució correcta*

		[['C'], ['E'], [], [], [], [], []] *[['A', 'X'], ['F', 'S', 'Z'], [], [], [], []]* *[['B'], [], [], [], []]* *[['B'], ['D'], [], ['U']]* *[['C'], ['E'], []]* *[['A', 'X'], ['F', 'S', 'Z']]* *[['B']]*	

b. Convertidor a CNF

A continuació s'han dissenyat jocs de prova per comprovar el funcionament del convertidor a CNF. Cada joc de prova es centra en validar una funcionalitat o un cas especial.

i. Prova 1

El primer joc de proves comprova que l'algoritme detecta un CNF; és a dir, que si la gramàtica d'entarda ja està en CNF no la canvia.

gramàtica inicial	***gramàtica CNF resultant***	***Observacions***
S > AA | AB | CC *A > BC | a* *B > b | a* *C > c*	*S > AA | AB | CC* *A > BC | a* *B > b | a* *C > c*	*No ha modificat la gramàtica perquè ja està en CNF*

ii. Prova 2

El següent joc de proves comprova que si alguna regla produeix el símbol inicial es crea un nou símbol inicial.

gramàtica inicial	***gramàtica CNF resultant***	***Observacions***
S > AA | AS *A > a*	S_0 *> AA | AS* *S > AA | AS* *A > a*	*Genera un nou símbol inicial correctament*

iii. Prova 3

Els següents jocs de prova pretenen comprovar el funcionament correcte del tractament de normes unitàries

gramàtica inicial	*gramàtica CNF resultant*	*Observacions*
$S > A$ $A > a \mid b$	$S > a \mid b$ $A > a \mid b$	*Identifica la norma unitària i la resol correctament*
$S > B$ $B > A \mid a$ $A > a \mid b \mid B$	$S > a \mid b$ $B > a \mid b$ $A > a \mid b$	*Es comprova que no afegeix dos cops la mateixa regla (no afegeix B > a perquè ja estava) i que no afegeix el mateix símbol a la producció (no afegeix B > B). Finalment també comprova que no es deixa cap producció unitària (quan substitueix la B de la S, també substitueix la A, no deixa S >A)*
$S > S \mid a$	$S > a$	*Comprova que elimina la regla S > S correctament.*
$S > AB$ $A > B$ $B > A \mid a$	$S > AB$ $A > a$ $B > a$	*Comprova que no va canviant eternament A i B, sinó que és capaç de resoldre-ho sense caure en un cicle infinit.*

iv. Prova 4

A continuació s'han creat jocs de prova per comprovar la funcionalitat d'eliminar les produccions de la paraula buida.

gramàtica inicial	*gramàtica CNF resultant*	*Observacions*
$S > AB$ $A > a$ $B > b \mid \varepsilon$	$S > AB \mid a$ $A > a$ $B > b$	*Detecta la producció de la paraula buida i és capaç de tractar-la*
$S > ABC$ $A > a \mid \varepsilon$ $B > b \mid \varepsilon$ $C > c \mid \varepsilon$	$S > X_0C \mid BC \mid AC \mid AB \mid \varepsilon$ $A > a$ $B > b$ $C > c$ $X_0 > AB$	*Es comprova que genera totes les produccions de S (BC, AC i AB). També comprova que s'eliminen totes les produccions de la paraula buida menys a la inicial, si en té.*
$S > ABA$ $A > a \mid \varepsilon$ $B > b$	$S > X_0A \mid AB \mid BA \mid b$ $A > a$ $B > b$ $X_0 > AB$	*Comprova que es té en compte totes les aparicions d'un símbol que produeix la paraula buida i es generen totes les possibles sub-produccions.*
$S > AB$ $A > AB \mid a$ $B > b \mid \varepsilon$	$S > AB \mid a$ $A > AB \mid a$ $B > b$	*Comprova que no afegeix la regla A > A per eliminar B. També comprova que no es torna afegir la mateixa regla dos cops (S > AB)*

$S > AD$ $A > BC$ $B > b \mid \varepsilon$ $C > c \mid \varepsilon$ $D > d$	$S > AD \mid d$ $A > BC \mid b \mid c$ $B > b$ $C > c$ $D\ d$	*Comprova que l'algoritme detecta que A genera la paraula buida directament i és capaç de tractar-la.*
$S > A$ $A > B$ $B > A \mid \varepsilon$	$S > \varepsilon$	*És capaç de tractar aquesta situació sense caure en un cicle infinit i sense deixar regles absurdes o incorrectes.*

v. Prova 5

Els següents jocs de prova s'han creat per comprovar el funcionament correcte del tractament de produccions amb més d'un símbol no terminal.

gramàtica inicial	***gramàtica CNF resultant***	***Observacions***
$S > ABC$ $A > a$ $B > b$ $C > c$	$S > X_0C$ $A > a$ $B > b$ $C > c$ $X_0 > AB$	*L'algoritme detecta la producció de més de dos símbols no terminals i la tracta creant una nova regla.*
$S > ABCBACB$ $A > a$ $B > b$ $C > c$	$S > X_4B$ $A > a$ $B > b$ $C > c$ $X_0 > AB$ $X_1 > X_0C$ $X_2 > X_1B$ $X_3 > X_2A$ $X_4 > X_3C$	*Comprova que l'algoritme genera regles noves fins que totes són binàries.*
$S > CBC$ $A > CBB$ $B > b$ $C > c$	$S > X_0C$ $A > X_0B$ $B > b$ $C > c$ $X_0 > cB$	*Comprova que si ha d'ajuntar un parell de símbols dels quals ja ha creat una regla no en crea un altra cop la mateixa.*

vi. Prova 6

Aquests jocs de prova es centren comprovar que el tractament de produccions mixtes (terminals i no terminals) és correcte.

gramàtica inicial	*gramàtica CNF resultant*	*Observacions*
$S > aA$ $A > b$	$S > X_0A$ $A > b$ $X_0 > a$	*L'algoritme detecta la producció de més de dos símbols no terminals i la tracta creant una nova regla.*
$S > aAa$ $A > aaB$ $B > b$	$S > X_0X_2$ $A > X_1B$ $B > b$ $X_0 > X_2A$ $X_1 > AX_2$ $X_2 > a$	*Comprova que torba tots els terminals de la producció i els separa correctament. També comprova que no crea dues normes per la mateixa producció.*

vii. Prova 7

Per últim s'ha creat un joc de proves per comprovar que a l'hora d'aplicar tots els passos alhora ho fa correctament.

gramàtica inicial	*gramàtica CNF resultant*	*Observacions*
$S > ASA \mid aB \mid C$ $A > B \mid \varepsilon$ $B > b \mid C\,C$ $C > c$	$S_0 > X_0A \mid X_1B \mid SA \mid AS \mid c$ $S > X_0A \mid X_1B \mid SA \mid AS \mid c$ $A > b \mid CC$ $B > b \mid CC$ $C > c$ $X_0 > AS$ $X_1 > a$	*L'algoritme és capaç de produir la gramàtica CNF següint tots els passos de forma correcta.*

c. PCKY

Primer de tot s'han provat els jocs de prova del CKY amb probabilitats trivials per assegurar que troba les regles correctament. A més, s'han provat nous jocs de prova per comprovar les noves funcionalitats.

i. Prova 1

El primer joc de proves pretén comprovar que l'algoritme calcula bé les probabilitats i assegurar-se de que si la paraula no és generada pel diccionari la probabilitat és zero. A més també comprova que es guardi bé la probabilitat de la paraula buida.

gramàtica:

$S > AA\ (0.7) \mid c\ (0.2) \mid \varepsilon\ (0.1)$

$A > a\ (0.2) \mid b\ (0.8)$

paraules: 'aa', 'ab', 'bb', 'c'

Paraula	***Booleà***	***Probabilitat***	***Taula PCKY***	***Observacions***
'aa'	*True*	*0.028*	*{'A': 0.2} {'S': 0.028} {'A': 0.2}*	*Calcula la probabilitat correctament 0.2*0.2*0.7=0.028*
'ab'	*True*	*0.112*	*{'A': 0.2} {'S': 0.112} {'A': 0.8}*	*Calcula la probabilitat correctament 0.2*0.8*0.7=0.112*
'bb'	*True*	*0.448*	*{'A': 0.8} {'S': 0.448} {'A': 0.8}*	*Calcula la probabilitat correctament 0.8*0.8*0.7=0.1448*
'c'	*True*	*0.(2)*	*{'S': 0.3}*	*Calcula la probabilitat correctament*

ii. Prova 2

El segon joc de proves s'encarrega de comprovar que quan hi ha el mateix símbol dos cops es una mateixa casella només guarda aquell que té la probabilitat més alta, assegurant-se també que guardi només un si tenen la mateixa probabilitat, i que si son un símbol diferent guardi els dos.

gramàtica:

S > AA (0.4) | BB (0.2) | BA (0.4)

X > AA (1)

A > a (1)

B > a (1)

paraules: 'aa'

Paraula	***Booleà***	***Prob***	***Taula PCKY***	***Observacions***
'aa'	*True*	*0.4*	*{'B': 1, 'A': 1} {'S': 0.4, 'X': 1} {'B': 1, 'A': 1}*	*Només guarda un de cada símbol i és el de major probabilitat*

iii. Prova 3

Finalment s'ha provat amb una gramàtica un poc més complexa per comprovar la capacitat de l'algorisme de calcular les probabilitats i guardar-les correctament.

gramàtica:

S > ZZ (0.35) | PQ (0.25) | UZ (0.25) | ε (0.15)

Z > ZZ (0.35) | PQ(0.15) | UZ (0.25) | b (0.25)

P > AA (0.6) | BB (0.4)

Q > CD(1)

U > QQ (0.75) | b (0.25)

A > a (1)

B > b (1)

C > c (0.9) | d (0.1)

D > d (1)

paraules: 'cdcdaa', 'aacdcdcdaacd'

Paraula	***Booleà***	***Prob***	***Taula PCKY***	***Observacions***
'cdcdaa'	*False*	*0*	*{'C': 0.9} {'Q': 0.9} {} {'U': 0.6075} {} {}* *{'D': 1, 'C': 0.1} {} {} {} {}* *{'C': 0.9} {'Q': 0.9} {} {}* *{'D': 1, 'C': 0.1} {} {}* *{'A': 1} {'P': 0.6} {}* *{'A': 1}*	*Reconeix que la gramàtica no genera la paraula*
'aacdcdcdaacd'	*True*	*0.00035*	*{'A': 1} {'P': 0.6} {} {'Z': 0.081, 'S': 0.135} {} {} {} {} {} {} {} {'S': 0.000349, 'Z': 0.000349}* *{'A': 1} {} {} {} {} {} {} {} {} {} {}* *{'C': 0.9} {'Q': 0.9} {} {'U': 0.6075} {} {} {} {} {} {}* *{'D': 1, 'C': 0.1} {} {} {} {} {} {} {} {}* *{'C': 0.9} {'Q': 0.9} {} {'U': 0.6075} {} {} {} {'Z': 0.0123, 'S': 0.0123}* *{'D': 1, 'C': 0.1} {} {} {} {} {} {}* *{'C': 0.9} {'Q': 0.9} {} {} {} {}* *{'D': 1, 'C': 0.1} {} {} {} {}* *{'A': 1} {'P': 0.6} {} {'Z': 0.081, 'S': 0.135}* *{'A': 1} {} {}* *{'C': 0.9} {'Q': 0.9}* *{'D': 1, 'C': 0.1}*	*Troba la solució correcta*

6. Conclusions

La pràctica realitzada ha permès implementar i entendre en profunditat l'algorisme CKY, tant en la seva versió determinista com en la probabilística i s'ha comprovat la seva eficàcia per verificar si una paraula pot ser generada per una gramàtica incontextual en Forma Normal de Chomsky (CNF).

A més, la implementació de les extensions ha millorat el projecte. El convertidor de CFG a CNF ha demostrat una transformació correcta, capaç de gestionar múltiples casos especials com produccions nules, unitàries o llargues, i produccions mixtes. Aquesta funcionalitat és essencial per assegurar la compatibilitat amb l'algorisme CKY.

L'algorisme CKY probabilístic ha aportat una funcionalitat addicional, permetent no només verificar la pertinença d'una paraula al llenguatge, sinó també calcular la seva probabilitat segons la gramàtica donada. Aquesta versió més avançada és molt útil en moltes àrees, com ara per fer models de llenguatge o anàlisi sintàctica estadística.

Finalment, els jocs de proves dissenyats han estat claus per validar totes les funcionalitats desenvolupades. Han permès identificar casos límit i assegurar que els algoritmes es comporten correctament en situacions diverses i complexes.

En conclusió, la pràctica ha assolit tots els objectius proposats, consolidant els coneixements de programació dinàmica i anàlisi sintàctic i dotant-nos d'eines sòlides per al tractament automàtic del llenguatge natural.